

9 - Data Scientist:

Feature

Engineer

ing

Feature Engineering

Introduction to Feature Engineering

Learn about feature engineering and its role in machine learning.

Introduction

Feature engineering is an integral part of building and implementing machine learning models. In this article, you'll learn about what feature engineering is, why we need it and where it fits in within the machine learning workflow. Additionally, you'll get a brief overview of the many data science tools involved in feature engineering and how they assist data scientists in model diagnostics.

What is feature engineering?

Before we can talk about feature engineering, we have to define what we mean by 'features'. A feature is a measurable property in a dataset, and a feature can be used as an input to a machine learning model. One way to think about features is as predictor variables that go into a model to predict the outcome variable. For example, if we want a model that predicts precipitation at a given place and time, we might use temperature, humidity, month, altitude, etc. as inputs. These are our features.

Often, when presented with a dataset, it might not be clear what features we should use for a specific model (i.e., should we use 'tree density' as an input to our precipitation model?). Similarly, in large datasets, there might be too many features to manually decide which features to use. Some features might be highly correlated with one another, some might not vary much with the outcome variable, and some might be in the wrong form to be a model input and so on. It is not uncommon that a data scientist might not realize any of this until they begin diagnosing a model that is performing poorly.

Feature engineering is a way to address these challenges. It is an umbrella term for the techniques we use to help us make decisions about features in machine learning model implementations.

Why do we need feature engineering?

To paraphrase Tolstoy, "All functional model implementations resemble one another but every dysfunctional model implementation is dysfunctional in its own way."

There are a lot of different ways that a model implementation can be dysfunctional. So the question is: What would make a model implementation functional or dysfunctional? Consider the following four attributes:

1. Performance:

We would like our machine learning model to perform "well" on our data. If it is not able to predict the outcome variable (to a reasonable degree of accuracy) on known data, it would be unwise to use it to predict outcomes on unknown data.

2. Runtime:

Suppose a model has excellent performance but takes a really long time to run. For a data scientist, depending on their available computational resources, such a model might be impractical for production.

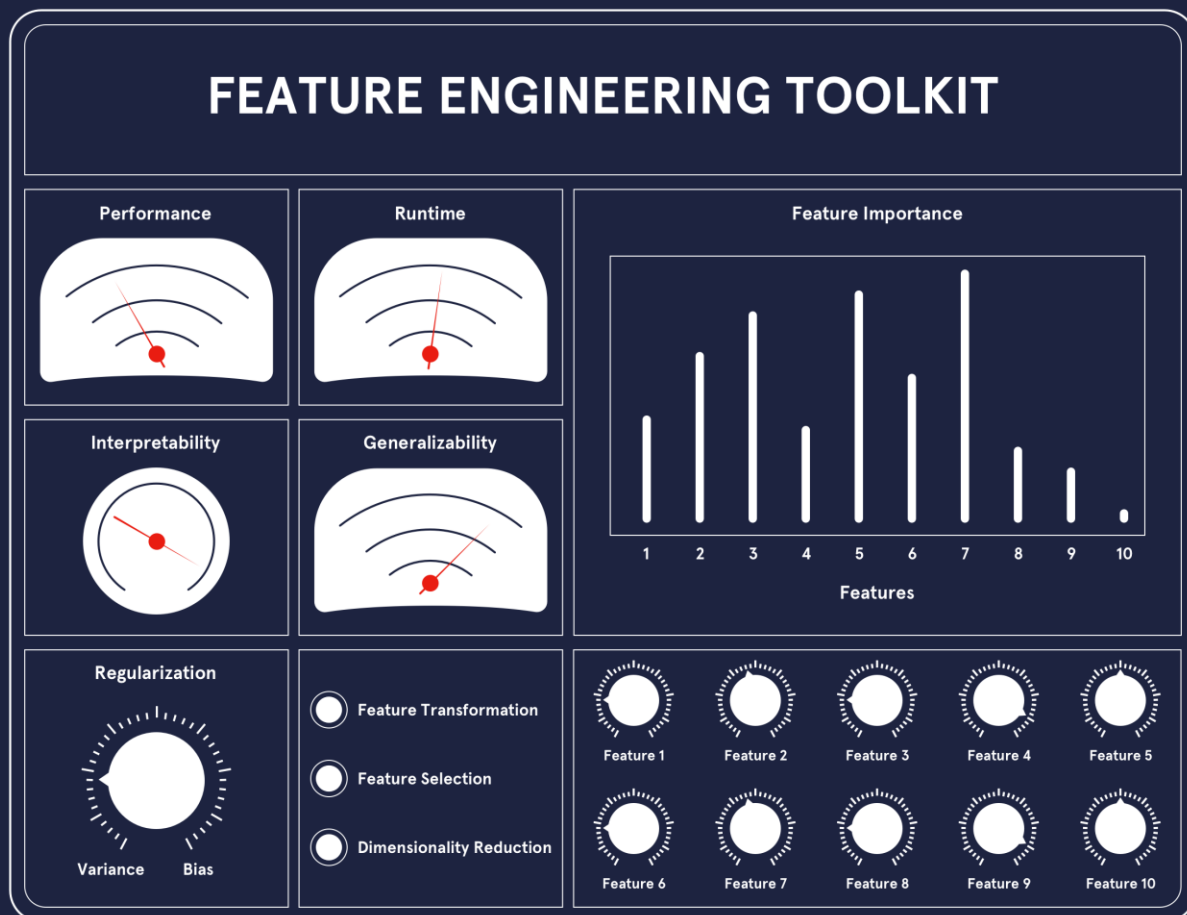
3. Interpretability:

A model is only as good as the insights it helps us glean from the data. Data scientists are often tasked with finding out what factors drive different outcomes. A well-performing model would not be of much help if it's opaque and uninterpretable.

4. Generalizability:

We would like our model to generalize well to unseen data. Often data scientists work with streaming data and need their model to be flexible with new and unknown data..

Feature engineering can be thought of as a toolkit of techniques to use when a model is missing one or more of the above attributes. If we imagine a model diagnostic machine (kind of like a modern version of this one) with meters representing the attributes, feature engineering would be akin to turning knobs and pushing buttons until we arrive upon a model that meets all of our attributes satisfactorily.



Feature Engineering and the Machine Learning Workflow

So where does feature engineering fall within the machine learning workflow? Does it go before modeling, after modeling or alongside modeling? The answer is: all of the above! Feature engineering is often introduced as an intermediate step between exploratory data analysis and implementing a machine learning algorithm. In reality, these distinctions are fuzzy and the process is not exactly linear.

Broadly, we can divide feature engineering techniques into three categories:

1. Feature Transformation methods:

These involve numerical transformations methods and ways to encode non-numerical variables. These techniques are applied *before* implementing a machine learning model. They include and are not limited to: scaling, binning, logarithmic transformations, hashing and one hot encoding. These methods typically improve performance, runtime and interpretability of a model.

2. Dimensionality Reduction methods:

Dimensionality of a dataset refers to the number of features within a dataset and reducing dimensionality allows for faster runtimes and (often) better performance. This is an extremely powerful tool in working with datasets with “high dimensionality”. For instance, a hundred-feature problem can be reduced to less than ten modified features, saving a lot of computational time and resources while maintaining or even improving performance. Typically, dimensionality reduction methods are machine learning algorithms themselves, such as Principal Component Analysis (PCA), Linear Discriminant Analysis (LDA), etc.

These techniques transform the existing feature space into a new subset of features that are ordered by decreasing importance. Since they “extract” new features from high dimensional data they’re also referred to as **Feature Extraction** methods. The transformed features do not directly relate to anything in the real world anymore. Rather, they are mathematical objects that are related to the original features. However, these mathematical objects are often difficult to interpret. The lack of interpretability is one of the drawbacks of dimensionality reduction.

3. Feature Selection methods

Feature selection methods are a set of techniques that allow us to choose among the pool of features available. Unlike dimensionality reduction, these retain the features *as they are* which makes them highly interpretable. They usually belong to one of these three categories:

i. Filter methods:

These are statistical techniques used to “filter” out useful features. Filter methods are completely model agnostic (meaning they can be used with any model) and are useful sanity checks for a data scientist to carry out before deciding on a model. They include and are not limited to: correlation coefficients (Pearson, Spearman, etc) , χ^2 , ANOVA, and Mutual Information calculations.

ii. Wrapper methods:

Wrapper methods search for the best set of features by using a “greedy search strategy”. They pick a subset of features, train a model, evaluate it, try a different subset of features, train a model again, and so on until the best possible set of features and most optimal performance is reached. As this could potentially go on forever, a stopping criterion based on number of features or a model performance metric is typically used. Forward Feature Selection, Backward Feature Elimination and Sequential Floating are some examples of wrapper method algorithms.

iii. Embedded methods:

Embedded methods are implemented *during* the model implementation step. Regularization techniques such as Lasso or Ridge tweak the model to get it to generalize better to new and unknown data. Tree-based feature importance is another widely used embedded method. This method provides insight into the features that are most relevant to the outcome variable while fitting decision trees to the data.

Summary

Here’s a brief summary of the feature engineering methods we’ve covered in this article, the attributes they seek to improve, and where they fit in within the machine learning workflow:

Feature Engineering Method	What	Why	Where
1. Feature Transformation	Transforms Numerical and non-numerical data	Performance, Runtime, Interpretability	Before model implementation
2. Dimensionality Reduction	Extracts a new feature subset	Runtime, Performance	Before model implementation
3. Feature Selection	Filter Methods	Interpretability, Performance	Before model implementation
	Wrapper Methods	Performance, Interpretability	Iterates over model implementation (i.e., before and after)
	Embedded Method: Regularization	Generalizability, Interpretability	Implemented with model implementation
	Embedded Method: Feature Importance	Interpretability, Performance	Implemented with model implementation

We're now ready to learn about these in-depth!

Data Transformations

Introduction: Data Transformations

Find out what you will learn in Data Transformations for Feature Analysis!

In this module you will learn how to prepare your data and features prior to running a machine learning model. This is a very essential step that, if skipped, can cause some grief down the line. It is usually performed right after or alongside Exploratory Data Analysis (EDA).

Why did we build this?

Let's take a moment to consider the consequence of not doing data transformations before a model implementation. An obvious one might be that the model code errors out. This is still a benign possibility as it tells you that something is wrong. The more insidious possibility is one where it doesn't cause a problem until much later. Multiple diagnoses of many poorly performing models later, it might become apparent that the data might've needed to be transformed in some way at the beginning! Data Transformations are thus an irreplaceable step for data scientists in their workflows.

What will you learn?

You're going to learn about:

- numerical data transformations such as standardizing, binning, scaling, etc.
- different ways of encoding categorical and non-numerical data.

when, where and how to apply these methods when presented with a dataset

Numerical Transformation Introduction

We've all heard it; a lot of work goes into getting your data just right for the task you have at hand. Perhaps you are trying to find insight into customer behavior or predicting the best time to send a marketing email. Whatever problem you may be trying to solve - you have probably already spent some time wrangling your data and getting it just right, and now you are at the stage where you need to prepare your data for machine learning.

For example, some machine learning models, like logistic regression and neural networks, can only handle numbers. Then random forest and decision tree models can take both numbers and text. We will call these text features categorical data, but more on that later. We also need to understand the spread of our data, as some models have a tough time handling data that has extreme outliers.

This process is called *numerical transformation*, when we take our numerical data and change it into another numerical value. This is meant to change the scale of our values or even adjust the skewness of our data. You may be thinking, "we already have our data in numbers. Why would we want to change those?" Well, first of all, that is a great question. We'll dive deep into the "why we do this" throughout this lesson. To put it simply, we do this to help our model better compare features and, most importantly, improve our model's accuracy and interpretability. That sounds like some good reasons to put the time and effort into numerical transformations if I do say so myself.

We'll focus on the following numerical transformations:

Centering
Standard Scaler
Min and Max Scaler
Binning
Log transformations

Let's get to know the data frame we will be using. This dataset has just over 100 responses from customers where they were asked about a recent Starbucks experience. You will soon notice that we have a mix of numerical and categorical data, but we'll focus only on the numerical features for this lesson.

Instructions

Checkpoint 1 Failed, try again

1.

We have provided you with a csv file under the name `starbucks_customers.csv` and have already imported pandas for you. Import the file and set it to a variable called `coffee`.

Hint

```
coffee = pd.read_csv('FILENAME.csv')
```

Copy to Clipboard

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Examine the features in your new data frame by printing the columns.

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Look at each feature within your data frame by printing `.info()`

```
import pandas as pd

## add code below
## set up dataframe
coffee = pd.read_csv('starbucks_customers.csv')

## print the column names
print(coffee.columns)

## get information on your data
print(coffee.info())
```

Centering Your Data

Data centering involves subtracting the mean of a data set from each data point so that the new mean is 0. This process helps us understand how far above or below each of our data points is from the mean.

We can look at the `nearest_starbucks` column.

```
distance = coffee['nearest_starbucks']
```

Copy to Clipboard

We will find the mean of our feature, create one line of code to center our data, and then plot the centered data. Here's what it will look like in Python.

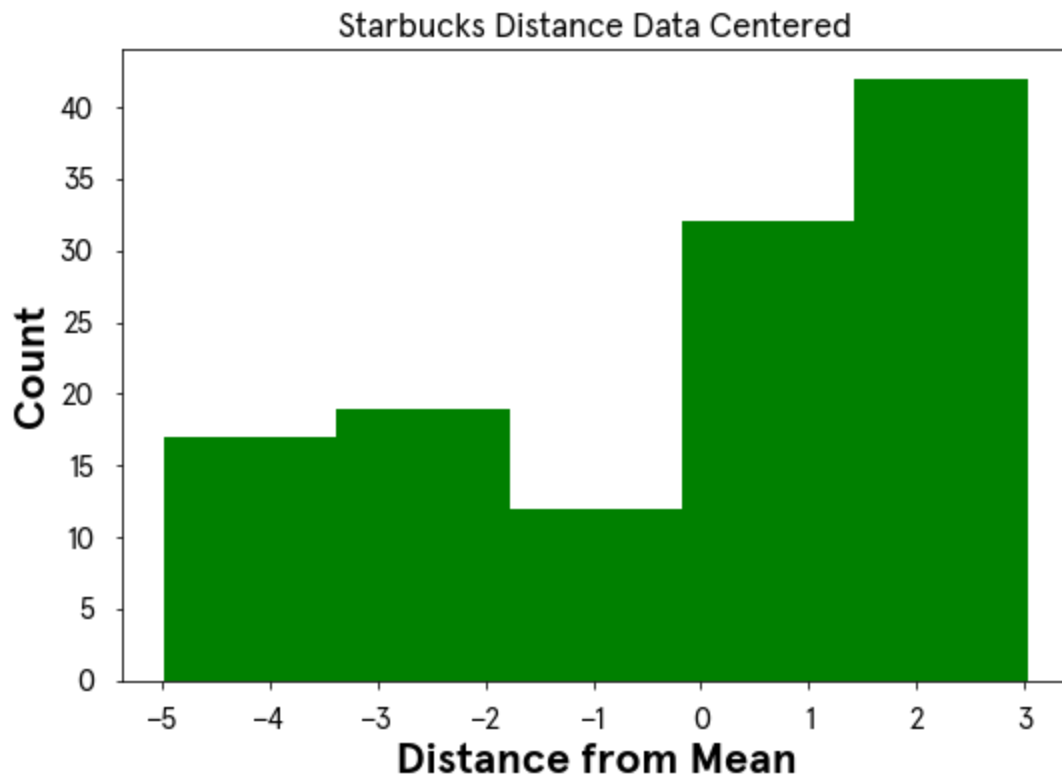
```
#get the mean of your feature
mean_dis = np.mean(distance)

#take our distance array and subtract the mean_dis, this will create a new series with
the results
centered_dis = distance - mean_dis

#visualize your new list
plt.hist(centered_dis, bins = 5, color = 'g')

#label our visual
plt.title('Starbucks Distance Data Centered')
plt.xlabel('Distance from Mean')
plt.ylabel('Count')
plt.show();
```

Copy to Clipboard



Now, look at that! Our data is centered at 0 and we can see the spread of data, many of our customers who took the survey tend to live fairly close to a Starbucks.

Let's try centering our age feature to see what happens there.

1.

Start by setting your `age` feature to a variable called `ages`

Hint

```
ages = df['column name']
```

Copy to Clipboard

Checkpoint 2 Passed

2.

It's helpful to know both the minimum and maximum age in our feature. First, create a variable called `min_age` and print the value

Hint

```
min_age = np.min(variable)
print(min_age)
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Now find the maximum age and set it to a variable called `max_age` and print the value.

Hint

```
max_value = np.max(feature)
print(max_value)
```

Copy to Clipboard

Checkpoint 4 Passed

4.

We want to better understand the spread of our data, so print the difference between `min_age` and `max_age`

Hint

Subtract the `min_age` from the `max_age`

Checkpoint 5 Passed

5.

We will need the mean age of our feature to center our data around, so create a variable called `mean_age` and print that value.

Hint

```
np.mean(variable)
```

Copy to Clipboard

Checkpoint 6 Passed

6.

Let's center our data! Set the result to a variable called `centered_ages` and print the results.

Hint

```
centered_dis = distance - mean_dis
```

Copy to Clipboard

Checkpoint 7 Passed

7.

Plot your centered data as a histogram.

Hint

```
plt.hist(centered_data)
plt.show()
```

Copy to Clipboard

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import codecademylib3_seaborn

coffee = pd.read_csv('starbucks_customers.csv')

## add code below
## create ages variable
ages = coffee['age']

## get min and print
min_age = np.min(ages)
print(min_age)

## get max and print
max_age = np.max(ages)
print(max_age)

## print the range
print(max_age - min_age)

## find the mean
mean_age = np.mean(ages)
print(mean_age)

## center ages
centered_ages = ages - mean_age
print(centered_ages)

## graph it
plt.hist(centered_ages)
plt.show();

plt.hist(coffee.age)
plt.show();
```

Standardizing our Data

Excellent work with centering the age feature from our dataset! Now we'll take that concept one step further and discuss standardizing our data. *Standardization* (also known as *Z-Score normalization*) is when we center our data, then divide it by the standard deviation. Once we do that, our entire data set will have a mean of zero and a standard deviation of one. This allows all of our features to be on the same scale. How cool is that?

This step is critical because some

Preview: Docs Machine learning is a branch of artificial intelligence that enables systems to learn from data and make predictions or decisions without explicit programming.

[machine learning](#)

models will treat all features equally regardless of their scale. You'll definitely want to standardize your data in the following situations:

- Before Principal Component Analysis

- Before using any clustering or distance based

- Preview: Docs Loading link description

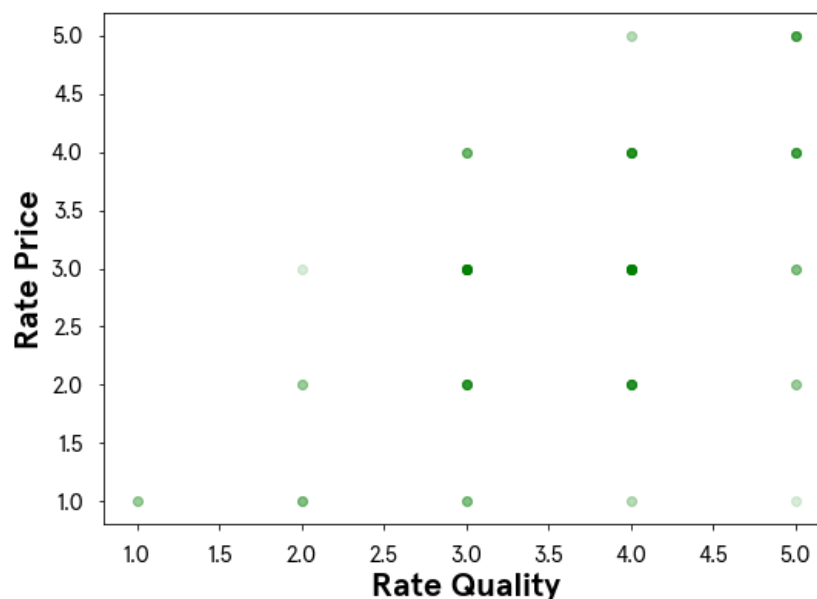
- [algorithm](#)

- (think KMeans or DBSCAN)

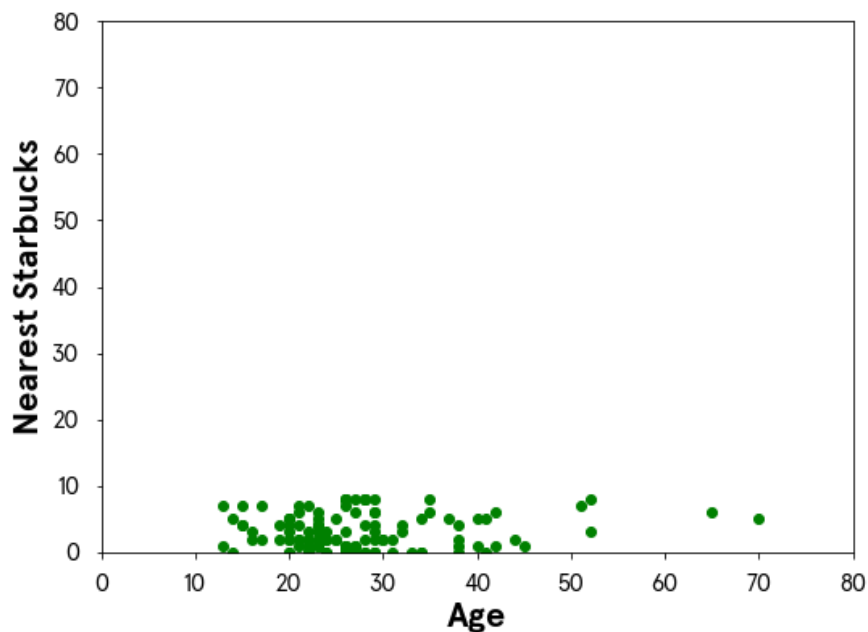
- Before KNN

- Before performing regularization methods like LASSO and Ridge

If I wanted to see how customers rated quality vs. price, I could graph a scatter plot of those two features and easily see that customers tend to score those two questions closely. Notice the darker spots (meaning more data points are laying on top of one another) around the 3.0 for "Rate Quality" and 3.0 for "Rate Price" as an example. This insight was easy to see since our features are on the same one to five scale.



Now, what happens if I am working with features on two totally different scales? Perhaps the "customer age" and "how far they live from a Starbucks" feature? Let's take a look.



Woah! Looking at this, it is much more challenging to gain insight or even identify patterns within our data. This will be a similar experience for our machine learning models. That's why when we standardize our entire dataset, we tend to see a more robust model performance if all features are on the same scale.

Let's examine one feature to learn the mathematics that goes on when we standardize our data. The mathematical formula will look like this:

$$z = (\text{value} - \text{mean}) / \text{stdev}$$

We'll look at just one feature, where customers were asked how close they are to their nearest Starbucks, and follow that formula above. First, we will set our `nearest_starbucks` feature to its own variable and then find the mean and standard deviation. Then we can quickly standard our list following the formula above.

```
distance = coffee['nearest_starbucks']

#find the mean of our feature
distance_mean = np.mean(distance)

#find the standard deviation of our feature
distance_std_dev = np.std(distance)

#this will take each data point in distance subtract the mean, then divide by the
standard deviation
distance_standardized = (distance - distance_mean) / distance_std_dev
```

Copy to Clipboard

We now have our distance feature standardized! Let's double-check by seeing what the mean and standard deviation of our [array](#) is.

```
# print what type distance_standardized is
print(type(distance_standardized))
#output = <class 'pandas.core.series.Series'>

#print the mean
print(np.mean(distance_standardized))
```

```
#output = 7.644158530205996e-17

#print the standard deviation
print(np.std(distance_standardized))
#output = 1.00000000000000013
```

Copy to Clipboard

Our outputs are basically mean = 0 and standard deviation = 1. Fantastic! Let's see what our `age` feature looks like standardized.

Instructions

Checkpoint 1 Passed

1.

Find the average of your variable `ages` and set the result to a variable called `mean_age`

Hint

```
average = np.mean(feature)
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Find the standard deviation of your variable `ages` and set the result to a variable called `std_dev_age`

Hint

```
standard_dev = np.std(feature)
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Standardize our age column. Set the result to a variable called `ages_standardized`

Hint

```
distance_standardized = (distance - mean_distance) / std_dev_distance
```

Copy to Clipboard

Checkpoint 4 Passed

4.

Print the mean and standard deviation of `ages_standardized`.

Hint

```
print(np.mean(lst))
print(np.std(lst))
```

```
import pandas as pd
import numpy as np

coffee = pd.read_csv('starbucks_customers.csv')
ages = coffee['age']

## add code below
## set up your variables
mean_age = np.mean(ages)
std_dev_age = np.std(ages)

## standardize ages
ages_standardized = (ages - mean_age) / std_dev_age

## print the results
print(np.mean(ages_standardized))
print(np.std(ages_standardized))
```

Standardizing our Data with Sklearn

Now that we understand the mathematics behind a standard scaler let's quickly implement it with the sklearn library. We will begin by importing our `StandardScaler` library from `sklearn.preprocessing`.

```
from sklearn.preprocessing import StandardScaler
scaler = StandardScaler()
```

Copy to Clipboard

We instantiate the `StandardScaler` by setting it to a variable called `scaler` which we can then use to transform our feature. The next step is to reshape our distance

Preview: Docs Loading link description

[array](#)

. `StandardScaler` must take in our array as 1 column, so we'll reshape our distance array using the `.reshape(-1,1)`

Preview: Docs A method is a small piece of code, usually defined in a class, that can be used outside the class and in other parts of the program.

[method](#)

. This numpy method says to take our data and give it back to us as 1 column, represented in the second value. The -1 asks numpy to figure out the exact number of rows to create based on our data. Pretty cool, right?

```
reshaped_distance = np.array(distance).reshape(-1,1)
distance_scaler = scaler.fit_transform(reshaped_distance)
```

Copy to Clipboard

Just like we learned in the last lesson, we do this so our data has a mean of 0 and standard deviation of 1. Let's print to see how the `StandardScaler` did.

```
print(np.mean(distance_scaler))
#output = -9.464196275493137e-17
print(np.std(distance_scaler))
#output = 0.9999999999999997
```

Copy to Clipboard

You'll notice the `e-17` at the end of our output for the mean, and that is our number written in exponential notation. Written in standard notation our answer is `-0.000000000000000009464196275493137`. Which basically has the mean at 0 and our standard deviation = 1. Great work! Now you can try to standardize the age column with the new knowledge you just gained.

Instructions

Checkpoint 1 Passed

1.

Instantiate `StandardScaler` by creating a variable named `scaler`

Hint

```
scaler = StandardScaler()
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Reshape our `ages` feature and set the result to a variable named `ages_resaped`

Hint

```
reshaped = np.array(feature).reshape(-1,1)
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Now we are ready to perform a `.fit_transform` on our `ages_resaped` feature and set the results to the variable named `ages_scaled`

Hint

```
feature_scaled = scaler.fit_transform(reshaped_feature)
```

Copy to Clipboard

Checkpoint 4 Passed

4.

Print the mean and standard deviation of your new variable `ages_scaled`

Hint

```
np.mean(feature_scaled)
np.std(feature_scaled)
```

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import StandardScaler

coffee = pd.read_csv('starbucks_customers.csv')
ages = coffee['age']

## add code below
## import standardscaler
scaler = StandardScaler()

## reshape
ages_resaped = np.array(ages).reshape(-1,1)

## fit_transform
ages_scaled = scaler.fit_transform(ages_resaped)

## print mean and standard deviation
print(np.mean(ages_scaled))
print(np.std(ages_scaled))
```

Min-Max Normalization

Another form of scaling your data is to use a min-max normalization process. The name says it all, we find the minimum and maximum data point in our entire data set and set each of those to 0 and 1, respectively. Then the rest of the data points will transform to a number between 0 and 1, depending on its distance between the minimum and maximum number. We find that transformed number by taking the data point subtracting it from the minimum point, then dividing by the value of our maximum minus minimum.

Mathematically a min-max normalization looks like this:

$$X_{norm} = (X - X_{min}) / (X_{max} - X_{min})$$

One thing to note about min-max normalization is that this transformation does not work well with data that has extreme outliers. You will want to perform a min-max normalization if the range between your min and max point is not too drastic.

The reason we would want to normalize our data is very similar to why we would want to standardize our data - getting everything on the same scale.

We'd do this process to our entire data set, but let's look at our distance to a nearest Starbucks feature. We already know from earlier in this lesson that our range in this feature is not too large, so it is a good candidate for a min-max normalization. We'll write a function that will perform the mathematics needed to transform the entire column.

```
distance = coffee['nearest_starbucks']

#find the min value in our feature
distance_min = np.min(distance)

#find the max value in our feature
distance_max = np.max(distance)

#normalize our feature by following the formula
```

```
distance_normalized = (distance - distance_min) / (distance_max - distance_min)
```

Copy to Clipboard

Excellent! Now if I were to print all the unique numbers in `distance_norm` here is what we will see.

```
{0.0, 0.125, 0.25, 0.375, 0.5, 0.625, 0.75, 0.875, 1.0}
```

Copy to Clipboard

We can see that all numbers are between 0 and 1. Amazing!

Instructions

Checkpoint 1 Passed

1.

We will now work with the `spent` feature in our data frame. Take just that column and set it equal to a new variable called `spent`.

Hint

```
new_variable = coffee['column_name']
```

Copy to Clipboard

Checkpoint 2 Passed

2.

Find the maximum amount someone spent and set that to a variable called `max_spent`

Hint

```
max_spent = np.max(variable)
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Find the minimum amount someone spent and set that to a variable called `min_spent`

Hint

```
min_spent = np.min(variable)
```

Copy to Clipboard

Checkpoint 4 Passed

4.

Find the difference between `max_spent` and `min_spent` and set that to a variable called `spent_range`. Print the results.

Hint

```
spent_range = max_spent - min_spent  
print(spent_range)
```

Copy to Clipboard

Checkpoint 5 Passed

5.

Normalize your `spent` column. Set the results to a variable called `spent_normalized`.

Hint

```
normalized = (feature - min_feature) / feature_range
```

Copy to Clipboard

Checkpoint 6 Passed

6.

Print your `spent_normalized` array to see how your numbers transformed.

Hint

```
print(array)
```

```
import pandas as pd  
import numpy as np
```

```

coffee = pd.read_csv('starbucks_customers.csv')

## add code below
## get spent feature
spent = coffee['spent']

#find the max spent
max_spent = np.max(spent)

#find the min spent
min_spent = np.min(spent)

#find the difference
spent_range = max_spent - min_spent
print(spent_range)

#normalize your spent feature
spent_normalized = (spent - min_spent) / spent_range

#print your results
print(spent_normalized)

```

Min-Max Normalization with Sklearn

Happy to see you now have a strong understanding of how a Min-Max Normalization works, let's try implementing it with the sklearn library.

We will start by importing our `MinMaxScaler` library from `sklearn.preprocessing`. Just like we covered in the `StandardScaler` exercise, we start by instantiating the `MinMaxScaler` by setting it to a variable called `mmscaler` which we can then use to transform our feature.

```

from sklearn.preprocessing import MinMaxScaler
mmscaler = MinMaxScaler()

```

Copy to Clipboard

The next step is to import our distance feature and reshape it so it is ready for our `mmscaler`.

```

#get our distance feature
distance = coffee['nearest_starbucks']

#reshape our array to prepare it for the mmscaler
reshaped_distance = np.array(distance).reshape(-1,1)

#.fit_transform our reshaped data
distance_norm = mmscaler.fit_transform(reshaped_distance)

#see unique values
print(set(np.unique(distance_norm)))
#output = {0.0, 0.125, 0.25, 0.375, 0.5, 0.625, 0.75, 0.875, 1.0}

```

Copy to Clipboard

Excellent. We now have a normalized feature within our dataset using the sklearn library `MinMaxScaler`!

Instructions

Checkpoint 1 Enabled

1.

Reshape the array, and set it to a variable called `spent_reshaped`

Stuck? Get a hint

Checkpoint 2 Step instruction is unavailable until previous steps are completed

2.

Instantiate MinMaxScaler and set the variable equal to `mmscaler`

Checkpoint 3 Step instruction is unavailable until previous steps are completed

3.

Now let's try using the MinMaxScaler from the imported sklearn library to perform a `.fit_transform` on our `spent_rescaled` feature and set the results to the variable `reshaped_scaled`

Checkpoint 4 Step instruction is unavailable until previous steps are completed

4.

Print the minimum and maximum number from your new normalized array nam

```
import pandas as pd
import numpy as np
from sklearn.preprocessing import MinMaxScaler

coffee = pd.read_csv('starbucks_customers.csv')
spent = coffee['spent']

## add code below
## reshape array
spent_rescaled = np.array(spent).reshape(-1, 1)

## instantiate mmscaler
mmscaler = MinMaxScaler()

## .fit_transform
reshaped_scaled = mmscaler.fit_transform(spent_rescaled)

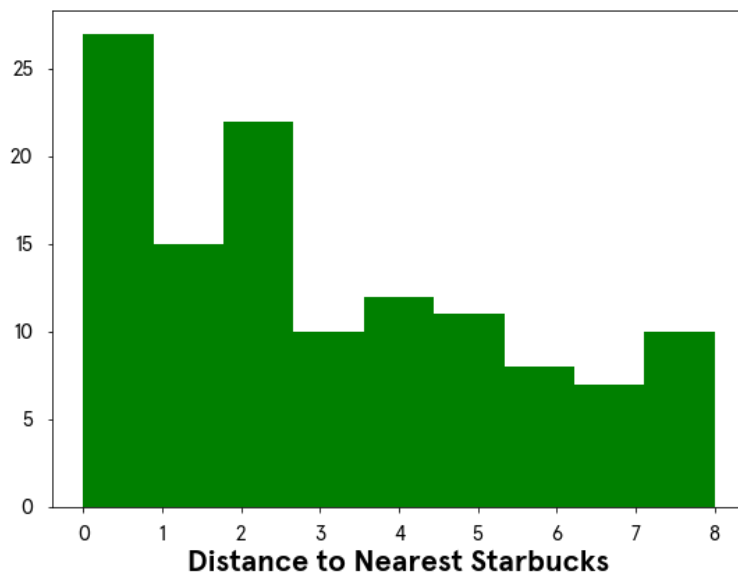
## print min and max
print(np.min(reshaped_scaled))
print(np.max(reshaped_scaled))
```

Binning our Data

Binning data is the process of taking numerical or categorical data and breaking it up into groups. We could decide to bin our data to help capture patterns in noisy data. There isn't a clean and fast rule about how to bin your data, but like so many things in machine learning, you need to be aware of the trade-offs.

You want to make sure that your bin ranges aren't so small that your model is still seeing it as noisy data. Then you also want to make sure that the bin ranges are not so large that your model is unable to pick up on any pattern. It is a delicate decision to make and will depend on the data you are working with.

Going back to our customer data and looking at our distance feature, let's look at the data with a histogram.



We can easily see that a lot of customers who completed this survey live fairly close to a Starbucks, and our data has a range of 0 km to 8km. I wonder how our data would transform if we were to bin our data in the following way:

```
distance < 1km
1.1km <= distance < 3km
3.1km <= distance < 5km
5.1km <= distance
```

First, we'll set the upper boundaries of what we listed above.

```
bins = [0, 1, 3, 5, 8.1]
```

Copy to Clipboard

Now you may be asking yourself 'Why end at 8.1? Isn't our max value 8?' That is true! We have 8.1 and not 8 because the pandas function we will use `pd.cut()` has a parameter where it will include the lower bound, and excludes the upper bound. Let me show you what I mean.

```
coffee['binned_distance'] = pd.cut(coffee['nearest_starbucks'], bins, right = False)
print(coffee[['binned_distance', 'nearest_starbucks']].head(3))
```

```
#output
#   binned_distance  nearest_starbucks
#0      [5.0, 8.1)                  8
#1      [5.0, 8.1)                  8
#2      [5.0, 8.1)                  8
```

Copy to Clipboard

We can see that those who marked 8 km now live in the `[5.0, 8.1)` bucket. The bracket `[` tells us 5.0 is included, and the parenthesis `)` tells us that 8.1 is excluded. We could write it as an inequality statement like this: `5 <= distance < 8.1` this allows our customers who marked 8 to belong to the 'Lives greater than 5.1 km' bin. Now let's have a look at our newly binned data.

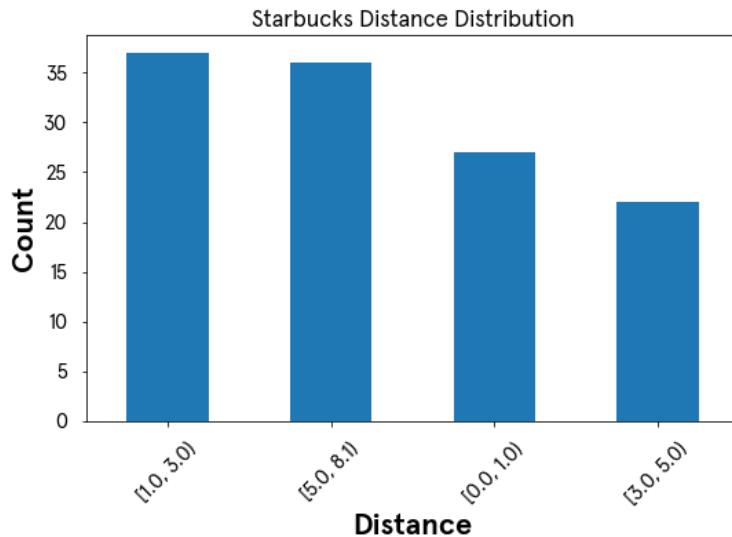
```
# Plot the bar graph of binned distances
coffee['binned_distance'].value_counts().plot(kind='bar')

# Label the bar graph
plt.title('Starbucks Distance Distribution')
```

```
plt.xlabel('Distance')
plt.ylabel('Count')

# Show the bar graph
plt.show()
```

Copy to Clipboard



Use this new skill to bin our age feature.

Instructions

Checkpoint 1 Passed

1. Print the age features min and max values.

Hint

```
print(np.min(feature))
print(np.max(feature))
```

Copy to Clipboard

Checkpoint 2 Passed

2. Based on our results from above, let's set the bin boundaries. Store it as a variable called `age_bins` and use the following limits [12, 20, 30, 40, 71]

Hint

Set the list provided to a variable called `age_bins`

Checkpoint 3 Passed

3. Create new binned column with `pd.cut()`. Name the new column `binned_ages`

Hint

```
df['new_column'] = pd.cut(df['original_col'], new_bins, right = False)
```

Copy to Clipboard

Checkpoint 4 Passed

4. Print the first 10 rows of the new column `binned_ages`

Hint

```
df['column_name'].head(10)
```

Copy to Clipboard

Checkpoint 5 Passed

5.

Plot a bar graph of new binned data.

Hint

```
df['binned_col'].value_counts().plot(kind='bar')
```

```
plt.show()
```

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import codecademylib3_seaborn

coffee = pd.read_csv('starbucks_customers.csv')
ages = coffee['age']

## add code below
## print min and max
print(np.min(ages))
print(np.max(ages))

## set bins
age_bins = [12, 20, 30, 40, 71]

## new column
coffee['binned_ages'] = pd.cut(ages, age_bins, right = False)

## print first 10 rows
print(coffee['binned_ages'].head(10))

## graph it
coffee['binned_ages'].value_counts().plot(kind='bar')
plt.show()
```

Natural Log Transformation

We've covered plenty of transformations! We have one last transformation we need to cover, and that is log transformations. Logarithms are an essential tool in statistical analysis and

Preview: Docs Loading link description

[machine learning](#)

preparation. This transformation works well for right-skewed data and data with large outliers. After we log transform our data, one large benefit is that it will allow the data to be closer to a "normal" distribution. It also changes the scale so our data points will drastically reduce the range of their values.

For example, let's explore a whole new data set from Kaggle around used car prices. Take a look at this histogram plot of 100,000 used car odometers.

```
import pandas as pd

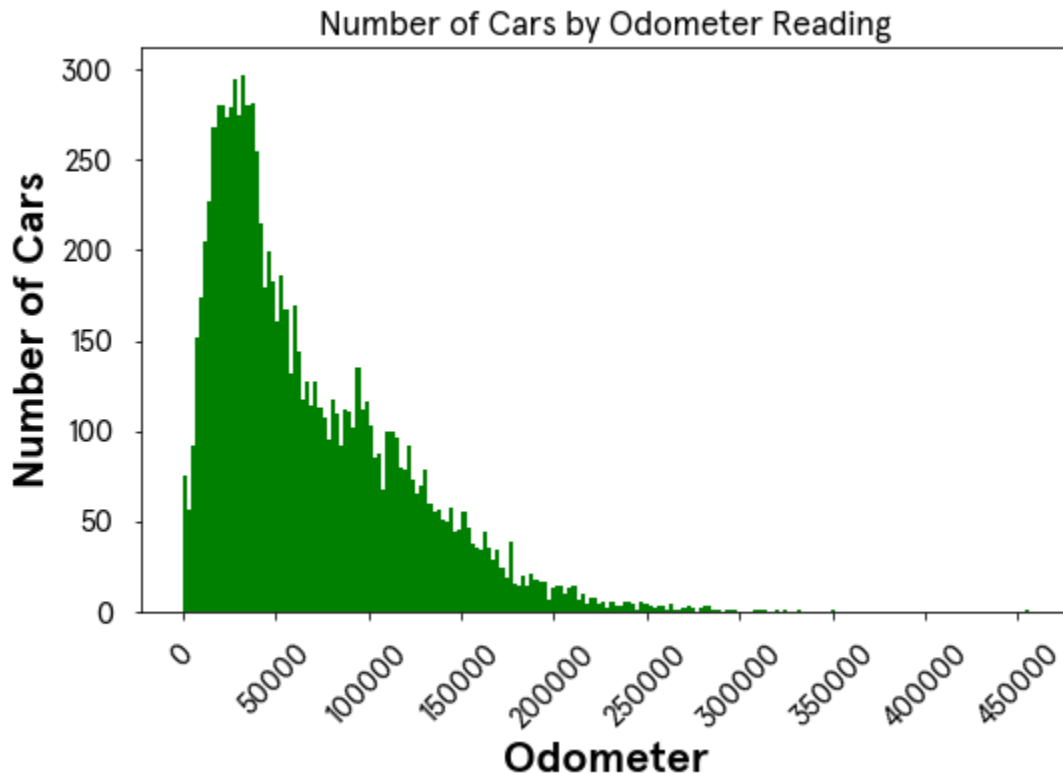
#import our dataframe
cars = pd.read_csv('cars.csv')

#set our variable
odometer = cars['odometer']
```

```
#graph our odometer readings
plt.hist(odometer, bins = 200, color = 'g')

#add labels
plt.xticks(rotation = 45)
plt.title('Number of Cars by Odometer Reading')
plt.ylabel('Number of Cars')
plt.xlabel('Odometer')
plt.show();
```

Copy to Clipboard



This histogram is right-skewed, where the majority of our data is located on the left side of our graph. If we were to provide this feature to our machine learning model it will see a lot of different cars with odometer readings off on the left of our graph. It will not see a lot of examples with very high odometer readings. This may cause issues with our model, as it may struggle to pick up on patterns that are within those examples off on the right side of our histogram.

We'll perform a log transformation using numpy to see how our data will transform.

```
import numpy as np

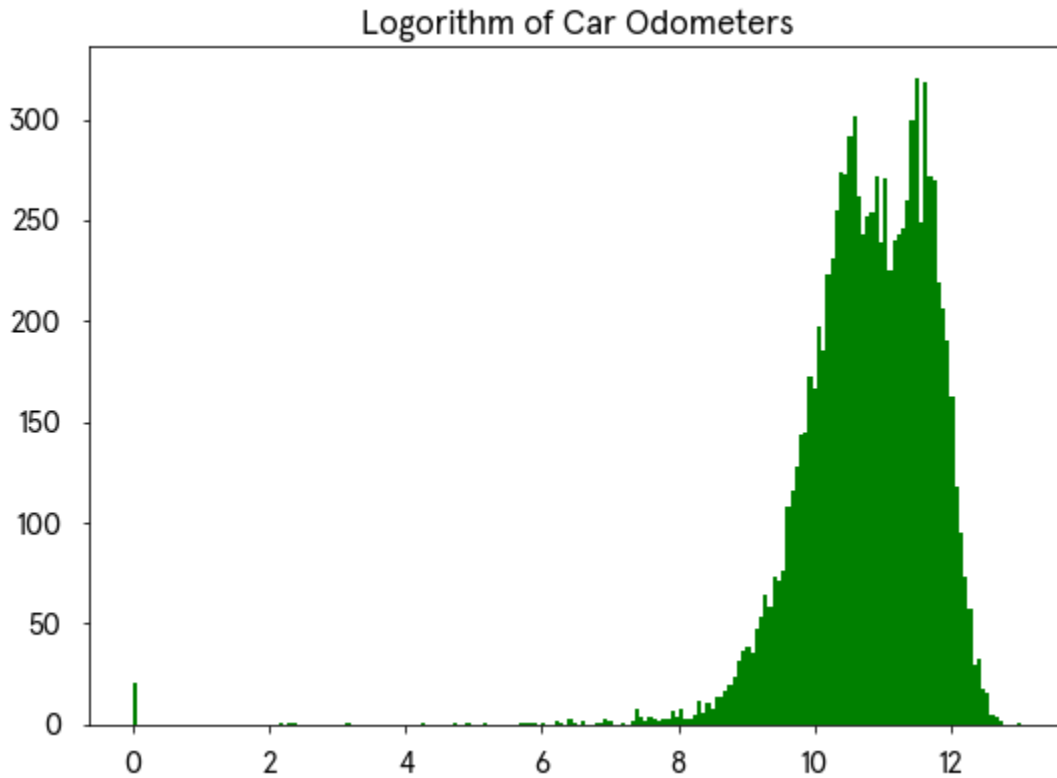
#perform the log transformation
log_car = np.log(cars['odometer'])

#graph our transformation
plt.hist(log_car, bins = 200, color = 'g')

#rotate the x labels so we can read it easily
plt.xticks(rotation = 45)
```

```
#provide a title
plt.title('Logarithm of Car Odometers')
plt.show();
```

Copy to Clipboard



Our data looks much closer to a normal distribution! If we were to look at a sample of five different cars with varying odometer readings, let's examine how the log transformation changed their values.

make	odometer	odometer_logged
Altima	10126	9.222862
Jetta	34042	10.435350
Camry	56762	10.946622
Civic	100103	11.513955

F-150	145695	11.889271
-------	--------	-----------

Saturn	151687	11.929574
--------	--------	-----------

If we compare the Altima with 10,126 miles to the Saturn with 151,687 miles those two cars have a huge difference in odometer readings. Yet, once we log transform the data we see the range from 9.22 to 11.93 is much smaller. Compressing the range of our data can help our model perform better!

There is so much more to add about log transformation. For the purpose of this exercise we just want to give a high overview and demonstrate how to log transform your data. Before we have you start testing your new skills let's quickly cover two major topics with log transformation:

Using a log transformation in a machine learning model will require some extra interpretation. For example, if you were to log transform your data in a linear regression model, our independent variable has a multiplication relationship with our dependent variable instead of the usual additive relationship we would have if our data was not log-transformed.

Keep in mind, just because your data is skewed does not mean that a log transformation is the best answer. You would not want to log transform your feature if:

- You have values less than 0. The natural logarithm (which is what we've been talking about) of a negative number is undefined.

- You have left-skewed data. That data may call for a square or cube transformation.

- You have non-parametric data

Instructions

Checkpoint 1 Passed

1.

We've imported all the necessary libraries, now you need to bring in the `cars.csv` file set it to a variable named `cars`.

Hint

```
variable = pd.read_csv('FILENAME.csv')
```

Copy to Clipboard

Checkpoint 2 Passed

2.

We will be working with the `sellingprice` feature in this data set, so set that column to a variable called `prices`.

Hint

```
variable = df['col_name']
```

Copy to Clipboard

Checkpoint 3 Passed

3.

Let's look at a histogram of `prices` to check to see if it is right-skewed. Set your histogram to show 150 bins so we can see more values in our graph.

Hint

```
plt.hist(feature, bins = 150)
plt.show()
```

Copy to Clipboard

Checkpoint 4 Passed

4.

Looks good! Now perform a log transformation on that variable `prices` and set the results to a variable called `log_prices`.

Hint

```
new_variable = np.log(variable)
```

Copy to Clipboard

Checkpoint 5 Passed

5.

Plot your newly log-transformed feature. Be sure to use `bins = 150`.

Hint

```
plt.hist(feature, bins = 150)
plt.show()
```

Encoding Categorical Variables

Encoding Categorical Variables

Encoding categorical variables with Python.

Introduction

Categorical data is data that has more than one category. When working with that type of data we have two types, nominal and ordinal. *Nominal data* is data that has no particular order or hierarchy to it, and *ordinal data* is categorical data where the categories have order, but the differences between the categories are not important or unclear.

We will be working with a dataset of used cars for this article to truly understand and demonstrate how to work with categorical data. Let's explore it and see what type of data we are working with.

```
import pandas as pd

# import data
cars = pd.read_csv('cars.csv')

# check variable types
print(cars.dtypes)
## OUTPUT
# year          int64
# make          object
# model         object
# trim          object
# body          object
# transmission  object
# vin           object
# state         object
# condition     object
# odometer      float64
# color         object
# interior      object
# seller        object
# mmr           int64
# sellingprice  int64
# saledate      object
```


We can see from the output that we have a lot of features that are `dtype = object` and that tells us those features could be text or a mix of text and numerical values. For our encoding examples, we will explore a few of those object features and transform those values so we can have a data frame ready for machine learning.

The reason we put our time into this level of encoding is that there are many machine learning models that cannot handle text and will only work with numbers. Our data must be encoded into numbers before we even begin to train, test, or evaluate a model.

Ordinal encoding

We mentioned already that *ordinal data* is data that does have order and a hierarchy between its values. Let us take a look at the `condition` feature from our data frame and perform a `value_counts` to see how many times each label is listed in our feature.

```
print(cars['condition'].value_counts())
# #OUTPUT
# New          2881
# Like New     2860
# Good         2027
# Fair         753
# Excellent    186
```

This is definitely an example of ordinal data: the condition of the used cars can easily be put in order of those in the “best” condition to the cars in the “worst” condition. The output printed the labels with the highest counts, but we can assume the following hierarchy:

We need to convert these labels into numbers, and we can do this with two different approaches. First, we can do this by creating a dictionary where every label is the key and the new numeric number is the value. ‘Excellent’ will get the highest score and ‘Fair’ will be our lowest score. Then we will map each label from the `condition` column to the numeric value and create a new column called `condition_rating`.

```
# create dictionary of label:values in order
rating_dict = {'Excellent':5, 'New':4, 'Like New':3, 'Good':2, 'Fair':1}

#create a new column
cars['condition_rating'] = cars['condition'].map(rating_dict)
```

The second approach we will show is how to utilize the `sklearn.preprocessing` library `OrdinalEncoder`. We follow a similar approach: we set our categories as a list, and then we will `.fit_transform` the values in our feature `condition`. We need to make sure we adhere to the shape requirements of a 2-D array, so you’ll notice the method `.reshape(-1,1)`.

We’ll also note, this method will not work if your feature has `NaN` values. Those need to be addressed prior to running `.fit_transform`.

```
# using scikit-learn
from sklearn.preprocessing import OrdinalEncoder

# create encoder and set category order
encoder = OrdinalEncoder(categories=[['Excellent', 'New', 'Like New', 'Good', 'Fair']])

# reshape our feature
condition_resaped = cars['condition'].values.reshape(-1,1)

# create new variable with assigned numbers
cars['condition_rating'] = encoder.fit_transform(condition_resaped)
```

Label Encoding

Now, we can talk about *nominal data*, and we have to approach this type of data differently than what we did with *ordinal data*. Our `color` feature has a lot of different labels, but here are the top five colors that appear in our data frame.

```
print(cars['color'].nunique())
# #OUTPUT
# 19

print(cars['color'].value_counts()[:5])
# #OUTPUT
# black      2015
# white      1931
# gray       1506
# silver     1503
# blue       869
```

To prepare this feature, we still need to convert our text to numbers, so let's do just that. We will demonstrate two different approaches, with the first one showing how to convert the feature from an `object` type to a `categories` type.

```
# convert feature to category type
cars['color'] = cars['color'].astype('category')

# save new version of category codes
cars['color'] = cars['color'].cat.codes

# print to see transformation
print(cars['color'].value_counts()[:5])
# #OUTPUT
# 2      2015
# 18     1931
# 8      1506
# 15     1503
# 3       869
```

Comparing our newly transformed data to the original top 5 list, we can see Black was transformed to number 2, White was transformed to 18, and so on.

However, we have created a problem for ourselves and potentially our model. We can see that 'Blue' cars now have a value of 3, and our model will assume that 'Blue' has lower precedence over the 'Black' car, whose color has a value of 2. Since 'Blue' cars = 3 and 'White' cars = 18, our model could actually give 'White' cars 6 times more weight than a 'Blue' car simply because of the way we

encoded this feature. To combat this ordinal assumption our model will make, we should *one-hot encode* our nominal data, which we will cover in the next section.

One more way we can transform this feature is by using `sklearn.preprocessing` and the `LabelEncoder` library. This method will not work if your feature has `NaN` values. Those need to be addressed prior to running `.fit_transform`.

```
from sklearn.preprocessing import LabelEncoder

# create encoder
encoder = LabelEncoder()

# create new variable with assigned numbers
cars['color'] = encoder.fit_transform(cars['color'])
```

One-hot Encoding

One-hot encoding is when we create a dummy variable for each value of our categorical feature, and a *dummy variable* is defined as a numeric variable with two values: 1 and 0. We will continue to talk about our `color` feature from our used car dataset.

Looking at this visual below, we can see we have ten cars in four different colors. In place of the single `color` column, we create four dummy variables - one new column for each color. Then the values that go into that column are binary, indicating if the car in that row is the color of the column name (1) or not (0).

Cars	Yellow	Green	Pink	Blue
	1	0	0	0
	0	1	0	0
	1	0	0	0
	1	0	0	0
	0	0	0	1
	0	1	0	0
	0	0	0	1
	0	0	1	0
	1	0	0	0
	0	1	0	0

This approach is great for our color feature and will allow the model to see each category as its own feature and not try to create order between a “Black car” and a “Red car”. Here is how we can implement this in Python:

```
import pandas as pd
# use pandas .get_dummies method to create one new column for each color
ohe = pd.get_dummies(cars['color'])

# join the new columns back onto our cars dataframe
cars = cars.join(ohe)
```

A downside to this approach is that it can create a lot of features which can then create a very sparse matrix.

Questions

One-hot encode the feature `body` from our cars dataset.

Hint

Binary encoding

If we find the need to one-hot encode a lot of categorical features which would, in turn, create a sparse matrix and may cause problems for our model, a strong alternative to this issue is performing a *binary encoder*. A *binary encoder* will find the number of unique categories and then convert each category to its binary representation. Let us take a quick review of binary numbers and keep using our `color` feature. We know that we have 19 unique colors, so the way to represent the numbers from 1 to 19 in binary format is as follows:

Number	Binary Number
1	1
2	10
3	11
4	100
5	101

6	110
7	111
8	1000
9	1001
10	1010
11	1011
12	1100
13	1101
14	1110
15	1111
16	10000
17	10001
18	10010
19	10011

We can easily see that our highest number 19 is 5 digits long, so our binary encoder will need 5 columns to be able to represent all digits. Here is a sample of how our `color` column will transform each color if we were to perform a binary encoder.

color_1	color_2	color_3	color_4	color_5	color_text
0	0	0	0	1	blue
0	0	0	1	0	—
0	0	0	1	1	gold
0	0	1	0	0	white
0	0	1	0	1	red
0	0	1	1	0	silver
0	0	1	1	1	black
0	1	0	0	0	gray
0	1	0	0	1	burgundy
0	1	0	1	0	brown
0	1	0	1	1	turquoise
0	1	1	0	0	orange
0	1	1	0	1	green
0	1	1	1	0	beige
0	1	1	1	1	yellow
1	0	0	0	0	purple
1	0	0	0	1	off-white
1	0	0	1	0	charcoal
1	0	0	1	1	pink

Our 19th color, pink, has transformed to be represented in the binary form 10011. If we were to utilize this process instead of the traditional one-hot encoder we would have 5 numerical features instead of 19, reducing our features by about 75%!

To make this happen with Python we'll use a library called `category_encoders` and import `BinaryEncoder`. We will determine which column to transform and set `drop_invariant` to `True` so it will keep the five binary columns. If it is set to the default 0, then we would have an additional column full of zeros.

```
from category_encoders import BinaryEncoder

#this will create a new data frame with the color column removed and replaced with our 5 new
binary feature columns
colors = BinaryEncoder(cols = ['color'], drop_invariant = True).fit_transform(cars)
```

Hashing

Another option we have available to us is an encoding technique called *hashing*. This process is similar to one-hot encoding where it will create new binary columns, but within the parameters, you can decide how many features to output. A huge advantage is reduced dimensionality, but a large disadvantage is that some categories will be mapped to the same values. That is called *collision*.

For example, we have 19 different colored cars. If I were to use the hash encoder and set the number of features to be 5, I will definitely have a few colors with the same hash values.

	col_0	col_1	col_2	col_3
color				
beige	0	1	0	0
black	0	0	0	1
blue	1	0	0	0
brown	0	0	1	0
burgundy	0	1	0	0
charcoal	0	0	1	0
gold	1	0	0	0
gray	0	1	0	0
green	0	0	0	0
off-white	0	0	1	0
orange	1	0	0	0
pink	1	0	0	0
purple	0	0	0	1
red	0	0	1	0
silver	0	1	0	0
turquoise	0	1	0	0
white	0	0	1	0
yellow	0	0	0	0
—	0	1	0	0

We can easily see that brown and charcoal colors have the same hash values. Meaning, we've lost some information and our model won't be able to see the difference between those two colors.

Here is how we can make this work with Python. Our final result of `hash_results` will produce a data frame of just 5 columns, so we will want to concatenate this new data onto our original data frame.

```
from category_encoders import HashingEncoder
```

```
# instantiate our encoder
encoder = HashingEncoder(cols='color', n_components=5)

# do a fit transform on our color column and set to a new variable
hash_results = encoder.fit_transform(cars['color'])
```

Now you may be thinking, when would I use this if I'm going to lose information and my model will see brown and charcoal (or some other color combo with the same hash value) as the same thing? Well, this could be a solution to your project and dataset if you are not as interested in assessing the impact of any particular categorical value.

For this example, maybe you aren't interested in knowing which color car had an impact on your final prediction, but you want to be able to get the best performance from your model. This encoding solution may be a good approach.

Target encoding

Target encoding is a Bayesian encoder used to transform categorical features into hashed numerical values and is sometimes called the *mean encoder*. This encoder can be utilized for data sets that are being prepared for regression-based supervised learning, as it needs to take into consideration the mean of the target variable and its correlation between each individual category of our feature. In fact, the numerical values of each category is replaced with a blend of the posterior probability of the target given a particular categorical value and the prior probability of the target over all the training data.

Woah, now that was a lot of Bayesian buzzwords. How would it work with our specific `color` feature? It replaces each color with a blend of the mean price of that car color and the mean price of all the cars. Had it been predicting something categorical, it would've used a Bayesian target statistic.

Some drawbacks to this approach are overfitting and unevenly distributed values that could lead to extremes. Let's review how to implement this in Python and check out what type of numerical values it will return. Again, we'll continue with our color feature - hope you are not yet tired of it!

Say we are preparing our dataset for a regression-based supervised learning algorithm that is trying to predict the selling price.

```
from category_encoders import TargetEncoder

# instantiate our encoder
encoder = TargetEncoder(cols = 'color')

# set the results of our fit_transform to a variable
# the output will be its own pandas series
encoder_results = encoder.fit_transform(cars['color'], cars['sellingprice'])

print(encoder_results.head())
#    color
# 0 11761.881473
# 1 18007.276995
# 2  8458.251232
# 3 14769.292595
# 4 12691.099747
```

We can examine all the different values our `encoder_results` holds, and if we look at the output from below we can see our min is about 3,054 and our max is about 18,048. That is quite a big difference!


```
# print all 19 unique values
print(np.sort(encoder_results['color'].unique()))
# OUTPUT
# [ 3054.12209927  8088.87434555  8458.25123153  9276.78571429
#   9867.50002121  9885.8093167   11043.90243902  11247.82608763
#   11761.88147296  11805.06187625  12124.83443709  12376.19047882
#   12691.09974747  13912.83399734  14769.29259451  15496.72704715
#   17174.36440678  17176.25931731  18007.27699531  18048.52540833]
```

Encoding date-time variables

Every data analyst or scientist will have to work with date-time objects at some point in their career. There is so much information to be gained from date-time objects. We will work with the `saledate` feature - yay, a new column to explore! The very first thing we need to do is convert this to a date-time object.

```
print(cars['saledate'].dtypes)
# # OUTPUT
# dtype('O')

cars['saledate'] = pd.to_datetime(cars['saledate'])
# #OUTPUT
# datetime64[ns, tzlocal()]
```

Now that we have our feature set up as a datetime object, let's demonstrate some methods we can utilize to get additional information.

```
# create new variable for month
cars['month'] = cars['saledate'].dt.month

# create new variable for day of the week
cars['dayofweek'] = cars['saledate'].dt.day

# create new variable for difference between cars model year and year sold
cars['yearbuild_sold'] = cars['saledate'].dt.year - cars['year']
```

Below are other options available through pandas `.dt`

Syntax	Description & Output
<code>df['col'].dt.year</code>	Outputs the year
<code>df['col'].dt.day</code>	Outputs the day number
<code>df['col'].dt.hour</code>	Outputs the hour from the time

`df['col'].dt.minute` Outputs the minute from the time

`df['col'].dt.second` Outputs the seconds from the time

`df['col'].dt.week` Outputs the week ordinal of the year

`df['col'].dt.dayofweek` Outputs the day of the week with Monday = 0 & Sunday = 6

Additional methods can be found [here](#).

Review

Wow, we covered a lot in this article! You have so many options when it comes to transforming categorical data into numerical values. We touched in:

- **Ordinal encoding**
- **Label encoding**
- **One-hot encoding**
- **Binary encoding**
- **Hashing**
- **Target encoding**
- **Date-time encoding**

We hope you can see the importance of getting your categorical features just right before you begin to train and test a model

Feature Importance

Learn about feature importance and how to calculate it.

Introduction

When we fit a supervised machine learning (ML) model, we often want to understand which features are most associated with our outcome of interest. Features that are highly associated with the outcome are considered more “important.” In this article, we’ll introduce you to the concept of **feature importance** through a discussion of:

[Gini impurity](#)

[scikit-learn](#)

Feature importance in an ML workflow

There are many reasons why we might be interested in calculating feature importances as part of our machine learning workflow. For example:

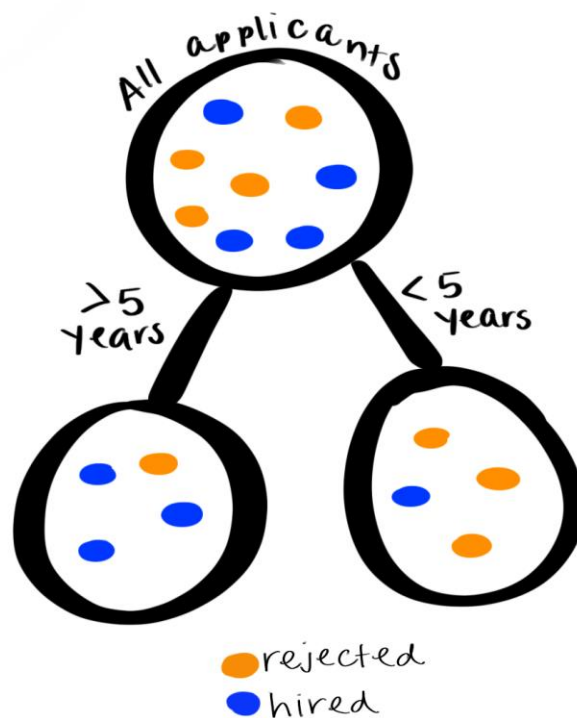
Calculating feature importance

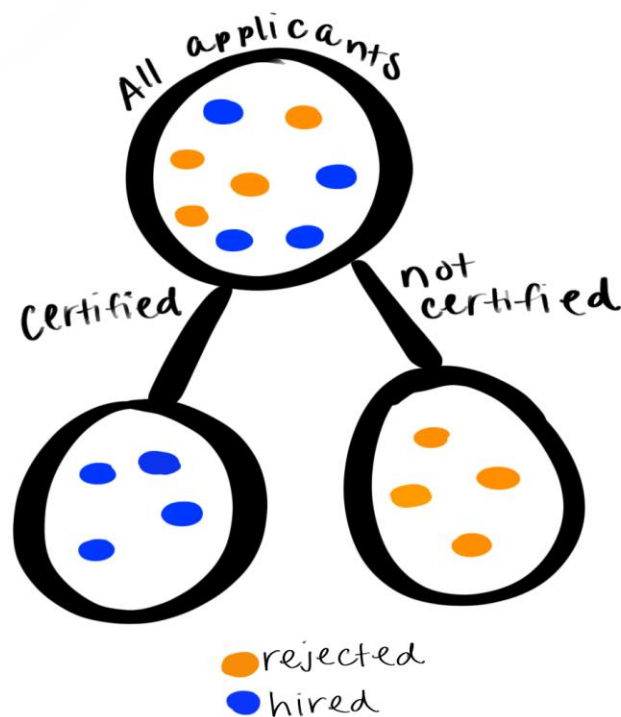
There are many different ways to calculate feature importance for different kinds of machine learning models. In this section, we'll investigate one tree-based method in a little more detail:

Gini impurity.

Gini impurity

Imagine, for a moment, that you're interested in building a model to screen candidates for a particular job. In order to build this model, you've collected some data about candidates who you've hired and rejected in the past. For each of these candidates, suppose that you have data on years of experience and certification status. Consider the following two simple decision trees that use these features to predict whether the candidate was hired:





Which of these features seems to be more important for predicting whether a candidate will be hired? In the first example, we saw that *most* candidates who had >5 years of experience were hired and *most* candidates with <5 years were rejected; however, *all* candidates with certifications were hired and *all* candidates without them were rejected.

Gini impurity is related to the extent to which observations are well separated based on the outcome variable at each node of the decision tree. For example, in the two trees above, the Gini impurity is higher in the node with all candidates (where there are an equal number of rejected and hired candidates) and lower in the nodes after the split (where most or all of the candidates in each grouping have the same outcome – either hired or rejected).

To estimate feature importance, we can calculate the Gini gain: the amount of Gini impurity that was eliminated at each branch of the decision tree. In this example, certification status has a higher Gini gain and is therefore considered to be more important based on this metric.

Gini importance in scikit-learn

To demonstrate how we can estimate feature importance using Gini impurity, we'll use the breast cancer dataset from `sklearn`. This dataset contains features related to breast tumors. The outcome variable is the diagnosis: either malignant or benign. To start, we'll load the dataset and split it into a training and test set:

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn import datasets

dataset = datasets.load_breast_cancer()
X = pd.DataFrame(dataset.data, columns=dataset.feature_names)
y = dataset.target
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.25)
```

Next, we'll fit a decision tree to predict the diagnosis using

`sklearn.tree.DecisionTreeClassifier()`. Note that we're setting `criterion= 'gini'`. This actually tells the function to build the decision tree by splitting each node based on the feature that has the highest Gini gain. By building the tree in this way, we'll be able to access the Gini importances later.

```
from sklearn.tree import DecisionTreeClassifier

clf = DecisionTreeClassifier(criterion='gini')

# Fit the decision tree classifier
clf = clf.fit(X_train, y_train)
```

Next, we can access the feature importances based on Gini impurity as follows:

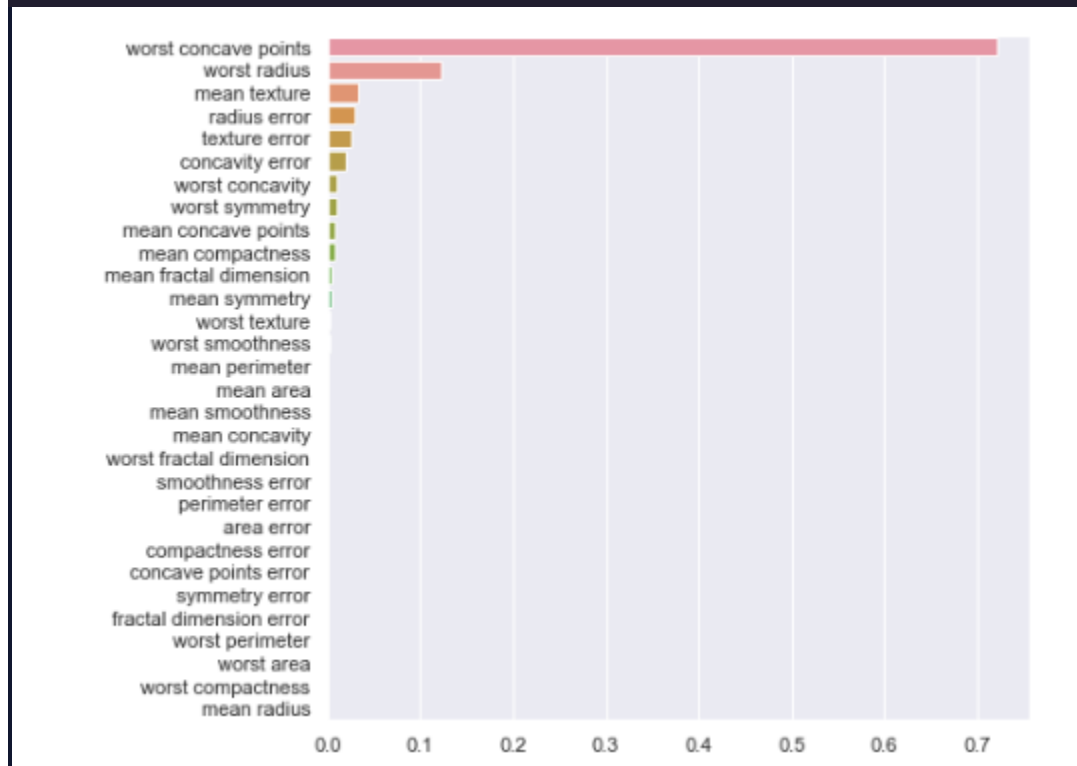
```
# Print the feature importances
feature_importances = clf.feature_importances_
```

Finally, we'll visualize these values using a bar chart:

```
import seaborn as sns
```

```
# Sort the feature importances from greatest to least using the sorted
indices
sorted_indices = feature_importances.argsort()[::-1]
sorted_feature_names = data.feature_names[sorted_indices]
sorted_importances = feature_importances[sorted_indices]

# Create a bar plot of the feature importances
sns.set(rc={'figure.figsize':(11.7,8.27)})
sns.barplot(sorted_importances, sorted_feature_names)
```



Based on this output, we could conclude that the features `worst concave points`, `worst radius` and `mean texture` are most predictive of a malignant tumor. There are also many features with importances close to zero which we may want to exclude from our model.

Pros and cons of using Gini importance

Because Gini impurity is used to train the decision tree itself, it is computationally inexpensive to calculate. However, Gini impurity is somewhat biased toward selecting numerical features (rather than categorical features). It also does not take into account the correlation between features. For example, if two highly correlated features are both equally important for predicting

the outcome variable, one of those features may have low Gini-based importance because all of its explanatory power was ascribed to the other feature. This issue can be mediated by removing redundant features before fitting the decision tree.

Other measures of feature importance

There are many other methods for estimating feature importance beyond calculating Gini gain for a single decision tree. We'll explore a few of these methods below.

Aggregate methods

Random forests are an ensemble-based machine learning algorithm that utilize many decision trees (each with a subset of features) to predict the outcome variable. Just as we can calculate Gini importance for a single tree, we can calculate average Gini importance across an entire random forest to get a more robust estimate.

Permutation-based methods

Another way to test the importance of particular features is to essentially remove them from the model (one at a time) and see how much predictive accuracy suffers. One way to "remove" a feature is to randomly permute the values for that feature, then refit the model. This can be implemented with any machine learning model, including non-tree-based methods. However, one potential drawback is that it is computationally expensive because it requires us to refit the model many times.

Coefficients

When we fit a general(ized) linear model (for example, a linear or logistic regression), we estimate coefficients for each predictor. If the original features were standardized, these coefficients can be used to estimate relative feature importance; larger absolute value coefficients are more important. This method is computationally inexpensive because coefficients are calculated when we fit the model. It is also useful for both classification and regression problems (i.e., categorical and continuous outcomes). However, similar to the other methods described above, these coefficients do not take highly correlated features into account.

Conclusion

In this article, we've covered a few different examples of feature importance metrics, including how to interpret and calculate them. We learned about:

`sklearn`

Feature importance is an important part of the machine learning workflow and is useful for feature engineering and model explanation, alike!

The What, When, and How of Dimensionality Reduction

Learn what dimensionality reduction is, when to use it, and common implementation methods.

The What, Why and How of Dimensionality Reduction

With the abundance of data available, we may find ourselves working with large datasets that contain hundreds or even thousands of features. While having a lot of data might seem like a good thing, too many features can actually become problematic.

The number of features is often referred to as the *dimensionality* of a dataset, and the set of challenges that come with working with high-dimensional data is notoriously dubbed the curse of dimensionality. In this article, we will take a closer look at the issues that can arise with high-dimensional data and some of the methods we can use to mitigate them.

The curse of dimensionality

In machine learning, there are two main issues with high dimensional data: *data sparsity* and *distance concentration*. In the following sections, we'll delve a little deeper into each one.

Data sparsity

Imagine we want to build a model that predicts whether a person will subscribe to an online newspaper based on two features: their browser type and the version of the webpage that they saw. Since we only have two features, all possible combinations of those features might look something like this:

Browser	Version	Subscribed
Chrome	A	1
Safari	A	0
Firefox	A	0
Chrome	B	1
Safari	B	1
Firefox	B	0

To successfully train a model that generalizes to new data, we want our training data to include all of these combinations (at least all of the ones that could exist in new data). For example, if the training dataset does not include any people who used a Chrome browser and saw version A of the website, we can't expect our model to make accurate predictions for that user. As we increase the dimension of the data, we can see how this quickly becomes a problem.

For example, just by adding one more feature, the total number of possible combinations increases exponentially:

Browser	Version	Source	Subscribed
Chrome	A	Email	1
Safari	A	Email	0
Firefox	A	Email	0
Chrome	B	Email	1
Safari	B	Email	1
Firefox	B	Email	0
Chrome	A	Ad	1
Safari	A	Ad	0
Firefox	A	Ad	1
Chrome	B	Ad	0
Safari	B	Ad	1
Firefox	B	Ad	0
Chrome	A	Google	0
Safari	A	Google	0

Firefox	A	Google	0
Chrome	B	Google	1
Safari	B	Google	0
Firefox	B	Google	0

Realistically, there will be a point when it is not feasible to obtain a large enough sample size to represent all possible combinations of features. Because a supervised model can only learn from what is contained in the training dataset, this can lead to overfitting and lack of generalizability.

Distance concentration

Another issue with high dimensional data is distance concentration. Many machine learning algorithms, such as k-nearest-neighbors and unsupervised clustering methods, involve calculating distances between observations. Observations that are “closer” to each other are clustered together — and are expected to have similar outcomes.

It turns out that, as the number of dimensions increases, the distance between all pairs of data points becomes increasingly similar. In other words, it becomes more difficult to discern whether two observations are any more or less similar to each other than to any other observation.

Dimensionality reduction techniques

So how can we mitigate these issues brought on by high-dimensional data? One approach is to use feature selection methods to retain only a certain subset of features in our dataset.

Another way to reduce dimensionality is to collapse the most relevant information across multiple features into a lower-dimensional space. We introduce three of the most common dimensionality reduction techniques below.

Principal component analysis (PCA)

Principal component analysis (PCA) is an unsupervised learning algorithm that transforms high-dimensional data into a smaller number of features using principal components (PCs). The goal is to preserve as much variance and information from the original data in a lower-dimensional space. The new features generated by PCA are linear combinations of the original features.

Linear discriminant analysis (LDA)

Like PCA, Linear discriminant analysis (LDA) can be used to transform data into new features that are linear combinations of the original ones. However, instead of maximizing the preserved variance from the original data, LDA works by maximizing the differences between known classes such that they are well separated in lower dimensional space.

T-distributed stochastic neighbor embedding (t-SNE)

T-distributed stochastic neighbor embedding (t-SNE) is another dimensionality reduction technique, but it uses a nonlinear method to map each data point from a high-dimensional space to a 2- or 3-dimensional space. As its name implies, t-SNE uses the student t-distribution to model the points in a way such that similar points are mapped closer to each other and dissimilar points are further apart. As a result, t-SNE is great at preserving the local structure of the original data.

Conclusion

Having a large number of features in a dataset can prove to be both a blessing and a curse. While there is the potential to contribute useful information, high-dimensional data also drastically increases the need for more data points in order to fill the space generated by the added dimensions. Otherwise, some machine learning algorithms can become ineffective or models might not be generalizable. Luckily, there are many dimensionality reduction techniques that can be used to mitigate these issues by transforming the data to a lower-dimensional space while still preserving the most relevant information from the original features.

Implementing PCA in Python

Exercise # 9: Review

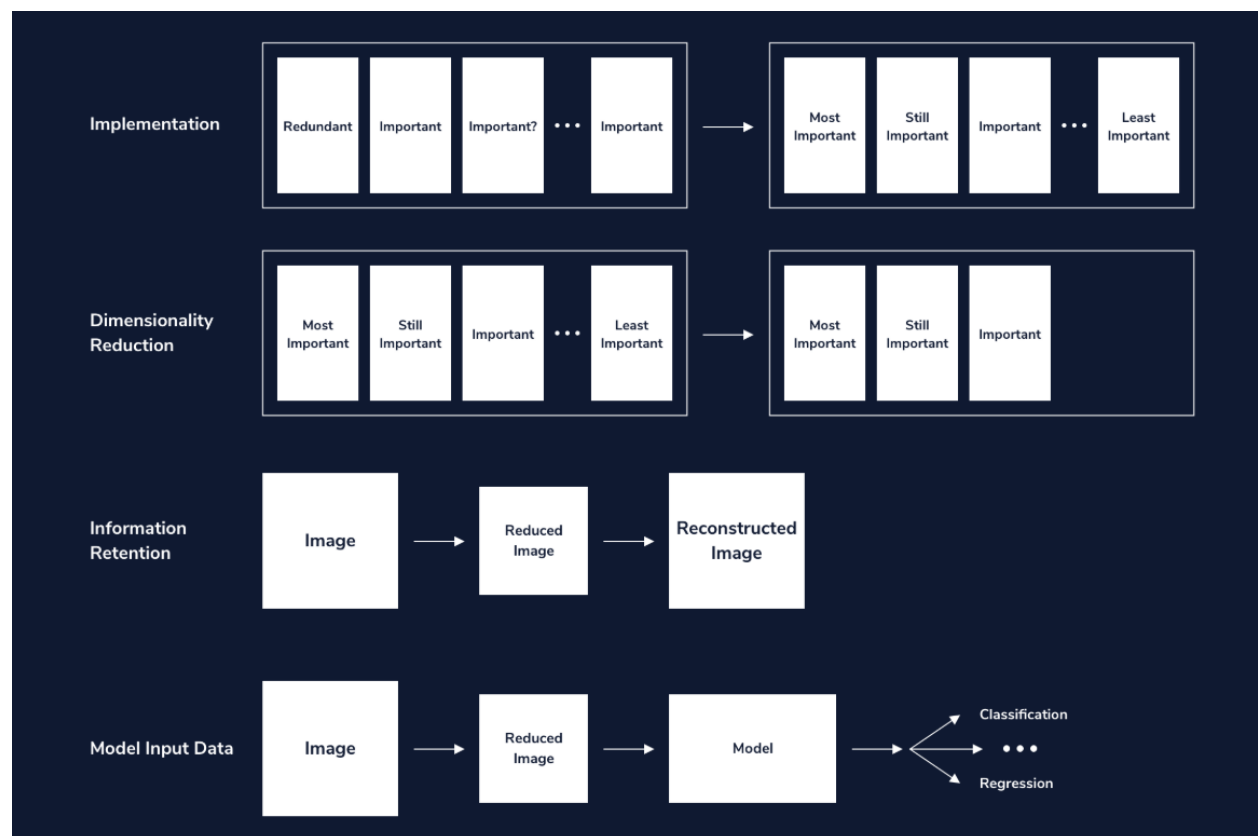
In this lesson, we have seen how PCA can be implemented using NumPy and scikit-learn. In particular, we have seen how:

Implementation: scikit-learn provides a more in-depth set of methods and attributes that extend the number of ways to perform PCA or display the percentage of variance for each principal axis.

Dimensionality reduction: We visualized the data projected onto the principal axes, known as principal components.

Image classification: We performed PCA on images of faces to visually understand how principal components still retain nearly all the information in the original dataset.

Improved algorithmic speed/accuracy: Using principal components as input to a classifier, we observed how we could achieve equal or better results with lower dimensional data. Having lower dimensionality also speeds the training



Linear Discriminant Analysis

Learn about the Linear Discriminant Analysis, a dimensionality reduction technique for classification problems, and how to implement it in `scikit-learn`.

Introduction

Linear discriminant analysis (LDA) is a dimensionality reduction technique used for classification problems with continuous independent variables. This technique can be used in a wide variety of applications including image recognition, marketing, biological classification, and more. It works by using linear algebra techniques to find a subspace within the space of independent variables that simplifies classification problems.

How LDA Works

Let's start with a simple example. Here's a classification problem with two continuous independent variables and two classes: a red class and a blue class.

In this example you could easily draw a line that would separate most of the red data points from the blue ones. But could you simplify the problem by reducing the data to a single

dimension? If we project the data onto a 1-dimensional subspace, we might be able to separate red from blue with only one dimension instead of two.

Here are two different ways of projecting the data onto 1-dimensional subspaces.

In the first example, there's no good way to separate red from blue. But in the second example you could easily choose a good decision boundary. This second subspace isn't just any subspace, it's actually the subspace that does the best job of separating red and blue. It was obtained by using LDA.

This is what LDA does in general. It finds the best possible subspace for a given classification problem. When there are only two classes, LDA finds a subspace that maximizes the ratio of variance between the classes to variance within the classes. This means that it finds a subspace where classes are far apart from each other, but the observations within each individual class are close to each other. LDA can also be applied when there are more than two classes, but this is slightly more complicated.

Once LDA finds the best subspace, data points can be projected onto that subspace. This yields a data set with fewer dimensions but nearly as much predictive power. LDA can also be used as a classifier by itself with the additional step of computing a decision boundary.

Notice that LDA is similar to Principal Component Analysis (PCA). Both LDA and PCA are dimensionality reduction tools. They both reduce dimensions in a similar way: by using linear algebra to find optimal subspaces for a statistical or machine learning problem.

A key difference between LDA and PCA is that they have different applications. PCA uses linear algebra techniques to find a subspace that maximizes the variance of a data set. This means that PCA basically finds the subspace that is best for linear regression problems. LDA, on the other hand, uses linear algebra techniques to find a subspace that is best for classification problems.

Multiple choice

Questions

Which of the following statements about Linear Discriminant Analysis (LDA) is FALSE?

Answer Choices

LDA can be used for both dimensionality reduction and classification.

When there are two classes, LDA maximizes the ratio of variance between classes to variance within classes.

LDA, like PCA, uses linear algebra to find a subspace of a given data set.

LDA can be used as an alternative to PCA when you have a continuous dependent variable.

Implementing LDA with scikit-learn

Let's take a look at how LDA performs in a real-life classification problem. The `seeds` data set comes from the [UCI Machine Learning Repository](#).

The original data set contains data about three varieties of wheat, but for the sake of simplicity we'll only look at the data for Kama and Rosa wheat. The problem is to classify wheat kernels based on their size and shape. We have seven independent variables to work with:

Scikit-learn's implementation of LDA allows us to do a few different things. First, we can transform the independent variables into a subspace with fewer dimensions. To create an LDA model that will find a 1-dimensional subspace you can use the code

`LinearDiscriminantAnalysis(n_components=1)`. Note that scikit-learn requires that `n_components` is less than the number of classes minus one and less than the number of features. Since this dataset has two classes, we can only set `n_components` to 1. After finding the 1-dimensional subspace we can use a classification algorithm.

Here's an example that uses LDA to find a 1-dimensional subspace of the seed data set.

```
# Import libraries
import pandas as pd
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

# Load data
seeds = pd.read_csv('seeds.csv')
X = seeds.drop('variety', axis=1)
y = seeds['variety']

# Create LDA model
lda = LinearDiscriminantAnalysis(n_components=1)

# Fit the data and create a subspace X_new
X_new = lda.fit_transform(X, y)
```

`X_new` is a 1-dimensional subspace of `x`. We can use it to train a classifier like logistic regression.

```
# Import library
from sklearn.linear_model import LogisticRegression

# Create logistic regression model
lr = LogisticRegression()

# Fit the model
lr.fit(X_new, y)
```

```
# Model accuracy
print(lr.score(X_new, y))
```

Output:

```
0.9928571428571429
```

As you can see, the model achieved near-perfect accuracy even though we reduced the data to a single dimension. For the sake of comparison, let's see how logistic regression performs on the original 7-dimensional data set.

```
# Create logistic regression model
lr_no_lda = LogisticRegression()

# Fit the model
lr_no_lda.fit(X, y)

# Model accuracy
print(lr_no_lda.score(X, y))
```

Output:

```
0.9428571428571428
```

In this case the model accuracy is still very good, but it happens to be a bit less accurate than the model that was trained on `x_new`. This illustrates the power of LDA. We were able to get similar results while greatly reducing the dimension of the data set.

Coding question

Questions

The beans data set comes from the [UCI Machine Learning Repository](#). The original data set contains data for seven different kinds of beans, but for the sake of simplicity you'll only look at two classes: barbunya beans and cali beans. The task is to classify beans based on their size and shape. There are 16 independent variables.

Use scikit-learn to create an LDA model called `lda`. Use the `fit_transform()` method to fit `lda` to `x` and `y` and create a 1-dimensional subspace called `x_new`.

Fit the logistic regression model to the new subspace `x_new` and `y`. Then create a variable called `lr_acc` and set it to the accuracy of the logistic regression model. You can use the `score()` method to find the accuracy. Then print `lr_acc`.

Code

1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17

Run

Run your code to check your answer

LDA as a classifier

We can also use LDA itself as a classifier. In this case, scikit-learn will project the data onto a subspace and then find a decision boundary that is perpendicular to that subspace. Put simply, it will do LDA and use the result to find a linear decision boundary.

Since we've already fit `lda` to `x` and `y`, we can simply look at the accuracy of the model by using the `score()` method.

```
print(lda.score(X, y))
```

Output:

```
0.9857142857142858
```

Coding question

Questions

Use LDA as a classifier on the beans data set. When using LDA as a classifier, it's not necessary to transform `x` into `x_new`. You can simply use the `fit()` method to fit `lda` to `x` and `y`. After fitting the model, create a variable called `lda_acc` and set it to the model's accuracy (`lda.score(X, y)`). Then print `lda_acc`.

Code

On this data set, LDA performs comparably to logistic regression.

Assumptions and caveats

As useful as LDA is, it relies on several assumptions. Some of these assumptions arise because LDA relies on linear algebra. In order for all the necessary linear algebra tricks to work out, a data set has to have some specific properties.

Class size versus the number of variables

LDA only works if the size of the smallest class is greater than the number of variables. If you have 10 variables and a class with only 8 observations, then LDA simply won't work. (The

technical reason for this is that LDA works by inverting a class's covariance matrix, and that matrix won't be invertible if you don't have more observations than variables.)

Normal variables

LDA relies on the assumption that independent variables in each class are multivariate normal. The technical definition of a multivariate normal distribution is that any linear combination of variables must be univariate normal (the univariate normal distribution is the familiar bell-shaped normal curve). Don't worry too much about the technical definition: in some applications it's fine to look at histograms of variables within each class and see if they look close enough to a bell curve.

Homoscedasticity

Homoscedasticity is the assumption that each class has the same covariance matrix. Intuitively this means that a scatter plot of one class has roughly the same size and shape as the scatter plot of every other class. If the values in one class are spread very far apart and the values in another class are clumped close together, or if the scatter plots of two classes are shaped like long, narrow ellipses that are perpendicular to one another, then LDA won't perform well. If, however, all scatter plots look similar, LDA can be applied. (Side note: if you compute the sample covariance matrices for each class in a data set, the matrices will almost never be exactly the same. It's okay if they're close enough.)

For cases when other assumptions hold true but the data is not homoscedastic, there is a technique called quadratic discriminant analysis (QDA). QDA is very similar to LDA, but it can be used when classes have different covariance matrices.

Other considerations

Another assumption of LDA is that independent variables are not multicollinear. This means that no variable is closely correlated with any other variable or linear combination of variables. This particular assumption is not as strict as the other assumptions, but it's good to be aware that LDA will generally perform worse on a data set with multicollinear variables.

Another consideration for LDA is that the observations in your data set should be statistically independent. If you have dependent variables (for example, multiple observations of the same subjects over time), LDA shouldn't be applied.

It's also important to note that LDA can be sensitive to outliers. If a data set has extreme outliers it may be best to remove them before doing LDA.

Finally, it's good to be aware that LDA is prone to overfitting. Scikit-learn's implementation of LDA addresses this by using a `shrinkage` parameter. This is a regularization technique for LDA that helps to prevent overfitting. We won't address this any further here, but it's something to be aware of if you use LDA in the future.

Multiple choice

Questions

Which of the following is NOT a required assumption of LDA?

Answer Choices

The size of the smallest class must be greater than the number of variables.

Data within each class should be multivariate normally distributed.

All class sizes should be equal.

Each class's covariance matrix should be (at least approximately) equal.

Conclusion

In summary, LDA is a powerful dimensionality reduction technique for classification problems with continuous independent variables. It uses linear algebra to find a subspace of a data set that does the best job of distinguishing between classes. It can also be used as a classifier. The scikit-learn library in Python allows you to easily implement LDA for both dimensionality reduction and classification. LDA does require certain assumptions about a data set, including multivariate normality, homoscedasticity, and the requirement that the size of the smallest class is greater than the number of variables.

